# Project 3: Supervised vs Unsupervised Anomaly Detection

*One-Class SVM, LOF, K-Means, DBSCAN, Isolation Forest, Autoencoders, PCA*

## 1 Objectives

In this lab, your goal is to build a **vanilla AI-based Intrusion Detection System** (IDS) from network connection logs. Each connection is labelled as normal or intrusion. For intrusions, the label is one of many types attack (DoS, Probe, R2L). A connection is described by its duration, network protocol, number of bytes sent, etc.

Throughout the lab, you will evaluate whether **unsupervised anomaly detection algorithms can automatically find anomalous patterns**, and how their performance changes with what you know about the data – e.g., knowing which points are normal vs having no knowledge at all. The lab follows this progression:

- **Domain-expert analysis:** before any algorithm, you will study how each attack differs from normal traffic using per-attack statistics of the features.
- **Shallow anomaly detection:** you will apply methods from different families – boundary-based (**One-Class SVM**), density-based (**LOF**), clustering-based (**K-Means**, **DBSCAN**), and isolation-based (**Isolation Forest**) – and explore what each of them considers an anomaly.
- **Deep anomaly detection:** you will train an **Autoencoder** on normal traffic and use its *reconstruction error* as an anomaly score.
- **Data representation:** you will compare a linear (i.e., **PCA**) and a non-linear (i.e., **Autoencoder bottleneck**) representation as input to the same detector, and visualise such representations with **t-SNE**.

Throughout this progression, you will examine **what each method considers anomalous** and whether this matches what we call an attack.

**Central question:** *How much do you need to know about normal traffic to detect attacks – and do the features of an attack always look like an anomaly?*

As in the previous labs, you will model and experimental choices: parameters, thresholds, and representations are not given. Some attempts may not work as expected. Analysing these results is part of the lab and can help you take further decisions.

Your final discussion should explain the choices you made, relate your conclusions to the results, and identify any limitations or open questions.

**A note on AI assistants.** You may use AI coding assistants to help you implement models, write training loops, or debug your code. However, you are responsible for understanding, checking, and validating the code you use. You must be able to identify and fix

bugs, including those in AI-generated code – this skill can be assessed as part of the exam. This requires knowing what the implementation should do and being able to recognise errors. You should be able to explain how your code works and why it is appropriate for the experiment you are carrying out.

The lab also requires you to make modelling choices and interpret your results. This includes choosing parameters and thresholds, deciding what each method should know about the data, and understanding why a method detects some attacks and misses others. AI assistants can support this work, but you need to assess their suggestions in the context of your data and experiments. Unexpected results and unsuccessful attempts are part of the learning process: discuss what you observed, what might explain it, and how it informed your next steps.

# 2 Lab Rules

Each student must join a self-organized group of 4 students. Each group submits **four zip files**, one per lab, named `groupID_labX.zip`. Each zip contains:

- **A report** (max 10 pages) describing your approach, decisions, experiments, and results, with tables and plots. Name it `groupID_labX.pdf`. The report is judged mainly on *reasoning* (see the rubric), not on raw performance numbers.
- **Your experiment logbook** (a section/appendix of the report or a separate sheet – see Section 4). This is required, and does not count in the number of pages.
- **The notebook(s) and code** you wrote.
  - *Best practice:* use Markdown headers and comments to (i) understand tomorrow what you did today and (ii) make us follow your solution.
  - **The notebook must be executed**: code *and* outputs must be visible.
  - **It must run** if we re-execute it. Set seeds where you rely on reproducibility.
  - **Format:** include the `.ipynb` and an HTML export.

Notice that this report **contributes to your examination** as described on the teaching portal. You might submit this report up to **10 days** before any exam session.

# 3 Dataset: Intrusion Detection System (IDS)

You will use **connection-level logs from a network-based Intrusion Detection System (IDS)**. IDSs monitor and analyse network traffic to detect unauthorised or malicious behaviour. They log metadata on each network connection, which is then used for offline training and evaluation of anomaly detection algorithms.

You are given two CSV files, `train.csv` and `test.csv` (the split is done for you):

- `train.csv`: labelled network connections, including normal traffic and several attack categories: Denial of Service (`dos`), Probing (`probe`), Remote-to-Local (`r2l`).
- `test.csv`: a separate set of connections, to assess whether your models *generalise*.

Each connection record is described by **features**, divided into three main categories:

- **Basic features**: connection duration, protocol type (e.g., TCP, UDP), service (e.g., HTTP, FTP), and status flags from the transport layer.

- **Content features**: information from packets' payload, e.g., the number of failed login attempts, access to sensitive files, or commands executed in the session.
- **Traffic features**: network-level statistics, e.g., number of connections to the same host or service in a time window to identify scanning and flooding behaviours.

The last two columns of each record are the attack **label** and the **binary label** (`0` for `normal` connections, `1` for all types of attacks, i.e., anomalies).

> ***The test set.*** *Use the test set* ***only where a task explicitly asks for it****. If you need a* ***validation set****, create it from* `train.csv` *- choosing and justifying an appropriate splitting strategy.*

# 4 Logbook, budget, hypotheses, and ablations

## 1. The experiment logbook (required deliverable)

Keep one logbook for the whole lab, report **only the training runs** (every fit of an anomaly detector, a clustering algorithm, or an Autoencoder).

| Run | Task | What I changed (one thing) | Hypothesis *before* running | Key result (val) | What I concluded |
|---|---|---|---|---|---|
| 1 | 2 | baseline: OC-SVM, rbf, normal data only, my choice of `nu` | knowing what normal looks like should give few false alarms | f1-macro 0.xx; few false alarms, some attacks missed | baseline established; some attacks look like normal traffic |
| 2 | 2 | normal only → all training data; `nu` adapted | anomalies in training will be enclosed by the boundary | f1-macro lower; more false alarms | confirmed; knowing normal data matters |

**Only runs that train a model go in the logbook.** Preprocessing and data-analysis decisions (Task 1) belong in your report text, not here. A diagnostic sweep used to choose a parameter (e.g., the SSE elbow for K-Means, the k-distance plot for DBSCAN, the explained variance for PCA) counts as **one** run.

- **Write the hypothesis *before* you run.** A wrong hypothesis you then explain is worth *more* than a vague right one. Do not rewrite history.
- **Change one thing per run.** If you change two things at once, you can't tell which one caused the effect.

## 2. The run budget

You have a budget of **∼25 logged training runs for the whole lab**. Spend them deliberately: before each run, your logbook must say *why this run is worth one of your remaining slots*. A rough guide: ∼7 for the OC-SVM (Task 2), a couple for LOF (Task 3), ∼4 for clustering (Task 4), a couple for Isolation Forest (Task 5), ∼5 for the Autoencoder (Task 6), ∼3 for the representations (Task 7). You do not have to use them all.

*The budget is the skill, not a hardware limit. In real settings every experiment costs money, time, and attention, and disciplined experimentation is what separates an engineer from someone trying things at random. "I ran a grid over every parameter" is **not** a good answer here.*

### 3. Expected and observed results

Before each experiment, **think about what you expect** to happen **and why**. Use this as a hypothesis to check against the results. You do not need to write down each hypothesis, but discuss any relevant or unexpected findings.

## Task 1 – The data from a domain-expert perspective

Inspect the data and report your findings.

1. **Explore the dataset.** Before preprocessing, explore the data.
   - **Q:** How many categorical and numerical attributes do you have?[1] How are the **attack labels** and the **binary label** distributed in the training and in the test set? What fraction of anomalies do you observe? Which implications for an anomaly detection task?
2. **Preprocessing.** Preprocess your features before applying any AI/ML algorithm.
   - **Q:** How do you preprocess categorical and numerical data? Where do you estimate the preprocessing parameters from?
3. **Explore your data from a domain-expert perspective.** In unsupervised learning, labels are typically unavailable, so domain experts rely on their knowledge of the application to interpret patterns in the data. For this exercise, you are given the attack labels to help you explore how feature values vary across attack types. Group the data points by attack label and create the following three heatmaps. In each heatmap, use rows for attack labels and columns for numerical features:
   - **Mean heatmap**: show the mean of each feature for each attack label.
   - **Standard deviation heatmap**: show the standard deviation of each feature for each attack label.
   - **Median heatmap**: show the median of each feature for each attack label.

   Include all three heatmaps in your report. **Q:** Comparing the heatmaps, which features, if any, appear to distinguish specific attack types? Consider both typical values (mean and median) and variability (standard deviation), and support your answer with examples from the heatmaps.

*No training runs are spent in this task. No need to track it in the logbook.*

[1] **Notice**: we consider 0/1 features as numerical.

# Task 2 – One-Class SVM: impact of the anomalies

Train a One-Class SVM (OC-SVM) with an `rbf` kernel to distinguish normal from anomalous traffic under the scenarios below. Labels may be used to construct the training subsets and evaluate predictions, but must not be used as a supervised signal.

- **Training on normal traffic only.** Assume that normal training samples can be identified. Remove all anomalous samples and train the OC-SVM using only normal traffic. Choose and justify a value of `nu`, and compare it with the default value, `nu=0.5`, keeping all other settings unchanged. Report the training false-positive rate and macro-averaged F1 score for both models.

  **Q:** Why is clean normal training data favourable for an OC-SVM? What does `nu` control, and how did you choose it? Can it be zero? How does your choice affect the fraction of normal training samples incorrectly flagged as anomalous?

- **Training on normal and anomalous traffic.** Assume that individual training labels are unavailable to the model, but the aggregate *contamination rate* (i.e., the percentage of anomalies on the total) may be known. Fix a random seed, and construct training subsets containing all normal training samples and respectively 0%, 10%, 20%, 50%, and 100% of the available anomalous training samples. For each subset, assume that its contamination rate is known and justify your choice of `nu`. For the 100% case, also train a model assuming unknown contamination: choose and justify `nu` without using the true contamination rate. Plot the training macro-averaged F1 score for each mixed subset.

  **Q:** How does knowing the contamination rate guide your choice of `nu`? What did you do when the contamination rate was unknown? How do the results change as more anomalies are included? Does knowing the contamination rate help in the 100% case? Is comparisons between scores on different training subsets a fair metric?

- **Generalisation to unseen traffic.** Evaluate the models trained on normal traffic only, with 10% of the available anomalies, and with 100% of the available anomalies on the same test set. Use the known-contamination setting.

  Report the binary macro-averaged F1 score and confusion matrix for each model. Also report detection recall separately for each attack type: the fraction of its samples correctly flagged as anomalous.

  **Q:** How do training and test results differ? Which model performs best on unseen traffic? Are errors mainly normal samples flagged as anomalous, or attacks classified as normal? Which attack types are most frequently missed? Suggest explanations using your earlier feature analysis.

# Task 3 – Local Outlier Factor (LOF)

LOF identifies each point's nearest neighbours using distances between feature vectors under a chosen distance metric. It uses these distances to estimate the point's *local density*: intuitively, how tightly packed the points are in its neighbourhood. LOF then compares this estimate with the local densities of the point's neighbours. A point is considered anomalous if its local density is substantially lower than theirs. The parameter `n_neighbors` controls how many neighbours are considered.

Use the training data, including anomalous traffic, to develop a LOF detector. Compare four values of `n_neighbors` and justify your selection criterion[2]. Then fit the selected configuration and evaluate it in the test set.

- **Q:** Also, can LOF detect the attacks? Explain whether and why the attack samples have lower local density than their neighbours. Also, discuss how `n_neighbors` affects their detection.

# Task 4 – Anomaly detection with clustering

Clustering algorithms do not look for anomalies directly: they group the data. You will derive anomalies from the clustering result.

## 4.A – K-Means

Fit K-Means on the full training data (normal + anomalous). Choose the number of clusters $k$ and justify your choice.

- **Q:** How big are the clusters? How are the attack labels distributed across the clusters? Are the clusters *pure* (i.e., do they consist of only one attack label)[3]?
- **Q:** How high is the silhouette per cluster? Are there clusters with a lower silhouette? If so, which attack labels are present in these clusters?
- **K-Means as an anomaly detector.** Use the distance of each point from its closest centroid as an anomaly score. If that distance is larger than a chosen threshold, you call the point an anomaly.

  **Q:** How do you select the threshold? Which types of attacks does this score detect, and which ones does it miss?
- **Visualising the representations with t-SNE.** Use t-SNE to obtain a 2D visualisation of the training set. Test 3 values of the perplexity[4]. Colour the points by attack **label**, and then with the **cluster ID**.

  **Q:** What differences do you see between the two visualizations? Which are the biggest overlaps, and why do you think it is the case? Are there types of attacks that fall in closer regions? Why does that happen?

## 4.B – DBSCAN

DBSCAN labels as *noise* (cluster $-1$) the points that do not belong to any dense region: we consider them anomalies. Choose **min_points** and $\epsilon$ and justify your choices.

- **Q:** Does the DBSCAN noise cluster consist only of **anomalous** points (cross-reference with the real attack labels)? How many clusters do you obtain?
- **Q:** Are there similarities with the LOF method you previously trained? Can you provide some examples?

[2] Analysing extremes is also a selection criterion

[3] Remember: this is something we can only measure in the lab assuming we know the labels. It is often NOT the case in the real world!

[4] Only from a visual point of view. See: https://distill.pub/2016/misread-tsne/

# Task 6 – Deep anomaly detection with Autoencoders

Now switch to Deep Learning. Deep Learning techniques can be used directly for anomaly detection or for data representation only.

- **Autoencoder – normal data only.** Create an Autoencoder. Choose the implementation details autonomously: the only requirement is that the architecture must have a shrinking encoder and an expanding decoder, with a bottleneck in between. Train it on normal data only – do the same on the validation set to choose the number of epochs and the learning rate. Now, focus on reconstruction errors: a point whose reconstruction error is higher than a threshold is an anomaly.

  **Q:** How do you estimate that threshold? Plot and report the ECDF of the reconstruction errors on i) the normal-only validation set, ii) the full training set, and iii) the test set. What can you say about the distribution of values? Where is higher? Why?

- **Autoencoder – normal data and anomalies.** Now, train another autoencoder on both normal data and anomalies. Report the results of this model and the previous one on the test set. Which of the two models perform best? Why do you think it is the case?

# Task 7 – PCA vs Autoencoder bottleneck

The bottleneck of an Autoencoder provides a lower-dimensional representation of the data, much like PCA. The key difference is that PCA represents the data through linear combinations of the original features, whereas an Autoencoder can learn nonlinear relationships between them. Compare the two representations using the same number of dimensions: does the Autoencoder's ability to capture nonlinear structure help preserve information that PCA misses?

- **Autoencoder bottleneck and OC-SVM.** Extract the encoder from the best model you choose on Task 6 and use it to compute the bottleneck embeddings of the normal training data. Train an OC-SVM on these embeddings, then use it on the embeddings of the test data.

- **PCA and OC-SVM.** Fit PCA on the normal training data only, and choose the number of components from the explained variance. Transform the training and test data, then train and evaluate an OC-SVM as above.

  **Q:** Compare the two OC-SVMs with the best original OC-SVM of Task 2 and with the Autoencoder based on the reconstruction error. Which is the best method overly? Are the Autoencoder's representation meaningful for anomaly detection?

# 5 Self-assessment Evaluation Process

*Lab 3 is graded. Use the following points to check your report and code before submission.* ***We value clear reasoning over high performance and do not compare results across students.*** *You must understand your code and be able to identify and fix bugs. We assess* ***YOUR understanding of the subject – not that of an LLM.***

| **Criterion** | **What you should provide** |
|---|---|
| Domain-expert analysis | Use the heatmaps to describe how each attack differs from normal traffic, and use these observations later to explain which attacks are detected or missed. |
| Parameter & threshold choices | Justify the parameters that turn a score into a decision (e.g., `nu`, `n_neighbors`, $k$, **min_points**, $\epsilon$, the Autoencoder threshold) using evidence from your data and plots, and state your threshold policy. |
| Knowledge assumptions | Clearly state what each method assumes to know about the data, keep labels for evaluation only, and discuss how the contamination of the training data affects the results. |
| Experimental approach | Use your expectations to guide experiments and compare them with the observed results. Explain the main changes you tried and what you learned from them, including unsuccessful attempts. Keep the logbook and justify how you used the run budget. |
| Representation choices | Compare the original features, PCA, and the Autoencoder bottleneck as input to the same detector, and use the t-SNE visualisations to discuss what each representation keeps or discards. |
| Interpretation of results | Support your explanations with your own plots, tables, and per-attack results. Discuss possible causes without assuming that a particular method must succeed or fail. |
| Comparison & complexity | Rank all methods on the validation set, considering performance, training time, and the knowledge each method requires, and explain whether the more elaborate approaches were useful for this dataset. |
| Code & reproducibility | Provide code that runs correctly and that you can explain, check, and debug. Keep training, validation, and test data separate in your experimental process. Document the settings needed to reproduce your results, including random seeds where relevant. |
| Report clarity | Provide a clear report within the page limit. Connect your conclusions to the experiments you performed and explain any limitations or unresolved questions. |